Interface Specification Doc – Salesforce (BD)

(ASR Loyalty)

Version	Date	Author	Description
1.0.0	30 January 2025	Rissaldo Antonio	Initial documentation

Contents

Interface Specification Doc – Salesforce (BD) 1

(ASR Loyalty) 1

Contents 2

Methods 3

1. Register 3

Request 3

Response 4

2. Check user existence 6

Request 6

Response 6

Glossary 7

Conventions 7

Encryption 7

Interface Specification AES-GCM - V2.1 8

Endpoint URLs 8

Creating t parameter 8

Methods

1. Register

Register new user as ASR member

Request

Method	URL
POST	api/asr_api/

Type	Params	Values
POST	sysToken	<sysToken>
POST	action	"register"
POST	salutation (optional)	String (1st letter Cap)
POST	firstName (optional)	string
POST	lastName (optional)	string
POST	displayName (optional)	string
POST	email(optional)*	string
POST	mobileNumber (optional)	string (+countrycode|mobileno) i.e. +65|86122031
POST	contactNumber (optional)	string (+countrycode|mobileno)
POST	gender (optional)	"M" or "F"
POST	dob (optional)	Date (datetime format)
POST	password(optional)*	string
POST	companyName (optional)	string
POST	companyEmailAddress (optional)	string
POST	countryOfResidence (optional)	ISO Alpha-2
POST	registrationType (optional)	['FULL','PARTIAL', 'BY_REFERRAL',' JOINANDBOOK'] (Default is 'FULL')
POST	offers (optional)	array ["M", "C"]
POST	referralCode (optional)	string (maximum 100 characters)
POST	euResident (optional)	string (either "yes" or "no)
POST	geoLocation (optional)	ISO Alpha-2

*Only for CPIS <systoken>: "email" and "password" are optional; "mobileNumber" is compulsory.

sysToken

`sysToken` must be sent with all client requests

CPRV will send email verification to user

ASR Password validation rule (Same as LYF)

- Minimum of 8, maximum of 16 characters
- Valid characters are alphanumeric and *!@#$%^&*()_+=-.*
- Must contain upper and lower case
- Must contains alpha and numeric characters

Registration type "PARTIAL" required fields by frontend are firstName, lastName, password, mobileNumber, email, salutation, countryOfResidence, offers.

mobileNumber field is required only when countryOfResidence is China.

Frontend will either pass in blank or both “M”&”C” for Offers field.

CPRV will send a different email verification to user where the text content is different from the one used for normal sign-up but the embedded link that redirects to brand website is the same. It must then call Update Profile to fill up the missing information.

Mapping table for offers
"M" – Member Offers
"C" – Corporate Offers
Both values must be passed in as required by Business to sign up both member and corporate offers.

List of salutation: Dr, , Mr, Mrs, Ms, Mx (1st letter must be in capital letter)

‘referralCode’ validation rule:

- Upper and/or lower case of alphabets A-Z
- Numbers 0-9
- Symbols: *!@#$%^*()_+.-*

Email validation rule:

- ([a-zA-Z0-9_\\.\\-\\&\\'*\\^\\~\\+\\=\\?\\{\\}])+\\@(([a-zA-Z0-9_\\-])+\\.)+([a-zA-Z0-9_]{2,4})+
- not more than 64 characters before the last "@"

Country List.xlsx

Member’s sign up auto-actication criteria:
If registrationType=”Full” and referral code have prefix GS20, then ‘auto-activated’ = Yes else No
If registrationType = “JOINANDBOOK” then ‘auto-activated’ = Yes

Response

Status	Response

200	{
	“status” : “success”,
	“auto-activated” : “Yes/No”
	}

200	{
	“status” : “success”,
	“memberId” : “1234567890” ,
	“auto-activated” : “Yes/No”
	}
	
	*Remark: this is only for registration type “PARTIAL”,“BY_REFERRAL” and “ JOINANDBOOK” that will return memberid

400	{
	“errorCode”:”1005”,
	“message”:[
	“ “Email is required.”, “Invalid email.”, “Email address is already registered!”
	“.”, “Mobile Number is already registered!”, “”, “Password is required.”,”Compar
	email is already registered”
	]
	}

400	{
	“errorCode”:”1006”,
	“message”:[
	“Referral Code consists of invalid character(s) or exceeded max char”
	]
	}

401	{
	"errorCode":"1502",
	"message":"Token is invalid."
	}

500	{
	"errorCode":"1800",
	"message":"Something went wrong. Please try again."
	}

2. Check user existence

Check whether the user exists and activated

Request

Method	URL
POST	api/asr_api/

Type	Params	Values
POST POST POST POST	sysToken action email mobileNumber	<sysToken> "checkUserExist" string string

- Either email or mobileNumber

Response

Status	Response

200	{
	"exist": "true",
	"activated": "true",
	"memberId": "12345678",
	"tierCode": "1"
	"accountType": “member” / “representative”
	
	}

400	{
	"errorCode":"1003"
	"message":"Email or mobile number is required"
	}

500	{
	"errorCode":"1800"
	"message":"Something went wrong. Please try again."
	}

Glossary

Conventions

- All responses are in JSON format.
- All request parameters are mandatory unless explicitly marked as [optional]

Encryption Interface Specification AES-GCM - V2.1

(ABW and Mobile API versions)

Endpoint URLs

ABW

UAT	https://ascott-uat.crmxs.com/?xs_app=asr_api_v2_1
PROD	**********

Creating t parameter

Version 2.1 requires constant values which CPRV will provide:

- PASSWORD
- SALT
- IV

UAT	
PASSWORD	%+h9>GbrU~K$7WGJvD6~wF$i;Z|{mru*IxXVRyg{)l*V#Xm^2x
SALT	sG3YrEDKPaj2nUXF
IV	PcGK5n4y72XL

**** PROD constants are to be provided separately***

Please refer to provided Classes:

- CryptoUtils.java – Utility class for EncryptorAesGcmPasswordV2.java. **No changes**.

```
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;
```

```java
import java.security.spec.KeySpec;
import java.util.ArrayList;
import java.util.List;

import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;

public class CryptoUtils {

	public static byte[] getRandomNonce(int numBytes) {
		byte[] nonce = new byte[numBytes];
		new SecureRandom().nextBytes(nonce);
		return nonce;
	}

	// AES secret key
	public static SecretKey getAESKey(int keysize) throws NoSuchAlgorithmException {
		KeyGenerator keyGen = KeyGenerator.getInstance("AES");
		keyGen.init(keysize, SecureRandom.getInstanceStrong());
		return keyGen.generateKey();
	}

	// Password derived AES 256 bits secret key
	public static SecretKey getAESKeyFromPassword(char[] password, byte[] salt)
			throws NoSuchAlgorithmException, InvalidKeySpecException {

		SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");
		// iterationCount = 65536
		// keyLength = 256
		KeySpec spec = new PBEKeySpec(password, salt, 65536, 256);
		SecretKey secret = new SecretKeySpec(factory.generateSecret(spec).getEncoded(), "AES");
		return secret;

	}

	// hex representation
	public static String hex(byte[] bytes) {
		StringBuilder result = new StringBuilder();
		for (byte b : bytes) {
			result.append(String.format("%02x", b));
		}
		return result.toString();
	}

	// print hex with block size split
	public static String hexWithBlockSize(byte[] bytes, int blockSize) {

		String hex = hex(bytes);

		// one hex = 2 chars
		blockSize = blockSize * 2;

		// better idea how to print this?
		List<String> result = new ArrayList();
		int index = 0;
		while (index < hex.length()) {
			result.add(hex.substring(index, Math.min(index + blockSize, hex.length())));
			index += blockSize;
		}

		return result.toString();

	}

}
```

- EncryptorAesGcmPasswordV2.java

The cause of slowness in V2 encryption and decryption is the creation of secretKey for each of the api payload. V2.1 eliminates this by making SALT constant thus secretKey creation is only once and can be reuse for all api payload's encryption and decryption.

Please refer V2.1 below:

```
import java.nio.ByteBuffer;
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import com.makena.utils.CryptoUtils;

public class EncryptorAesGcmPasswordV2 {

    private static final String ENCRYPT_ALGO = "AES/GCM/NoPadding";

    public static final int TAG_LENGTH_BIT = 128; // must be one of {128, 120, 112,
104, 96}
    public static final int IV_LENGTH_BYTE = 12;
    public static final int SALT_LENGTH_BYTE = 16;
    public static final Charset UTF_8 = StandardCharsets.UTF_8;

    private String salt_S                              = "";
    private String iv_S                                = "";
    private SecretKey aesKeyFromPassword_S = null;

    public String getSalt_S() {
          return salt_S;
    }

    public void setSalt_S(String salt_S) {
          this.salt_S = salt_S;
    }

    public String getIv_S() {
          return iv_S;
    }

    public void setIv_S(String iv_S) {
          this.iv_S = iv_S;
    }

    public SecretKey getAesKeyFromPassword_S() {
          return aesKeyFromPassword_S;
    }

    public void setAesKeyFromPassword_S(SecretKey aesKeyFromPassword_S) {
          this.aesKeyFromPassword_S = aesKeyFromPassword_S;
    }

    public String encrypt(byte[] pText, String password) throws Exception {
```

```
        // 16 bytes salt
        //byte[] salt = CryptoUtils.getRandomNonce(SALT_LENGTH_BYTE);
          byte[] salt = this.salt_S.getBytes();


        // GCM recommended 12 bytes iv?
        //byte[] iv = CryptoUtils.getRandomNonce(IV_LENGTH_BYTE);
          byte[] iv = this.iv_S.getBytes();

        // fail-safe if secretKey is not set
          if(aesKeyFromPassword_S == null){
                aesKeyFromPassword_S =
CryptoUtils.getAESKeyFromPassword(password.toCharArray(), salt); // fail-safe
                System.out.println("Creating SecretKey...");
          }else {
                System.out.println("Reusing SecretKey...");
          }


        Cipher cipher = Cipher.getInstance(ENCRYPT_ALGO);

        // ASE-GCM needs GCMParameterSpec
        cipher.init(Cipher.ENCRYPT_MODE, aesKeyFromPassword_S, new
GCMParameterSpec(TAG_LENGTH_BIT, iv));

        byte[] cipherText = cipher.doFinal(pText);

        // prefix IV and Salt to cipher text
        byte[] cipherTextWithIvSalt = ByteBuffer.allocate(iv.length + salt.length +
cipherText.length)
                .put(iv)
                .put(salt)
                .put(cipherText)
                .array();

        // string representation, base64, send this string to other for decryption.
        return Base64.getEncoder().encodeToString(cipherTextWithIvSalt);

    }

    // we need the same password, salt and iv to decrypt it
    public String decrypt(String cText, String password) throws Exception {

        byte[] decode = Base64.getDecoder().decode(cText.getBytes(UTF_8));

        // get back the iv and salt from the cipher text
        ByteBuffer bb = ByteBuffer.wrap(decode);

        byte[] iv = new byte[IV_LENGTH_BYTE];
        bb.get(iv);

        byte[] salt = new byte[SALT_LENGTH_BYTE];
        bb.get(salt);

        byte[] cipherText = new byte[bb.remaining()];
        bb.get(cipherText);

        // fail-safe if secretKey is not set
```

```
        if(aesKeyFromPassword_S == null){
                aesKeyFromPassword_S =
CryptoUtils.getAESKeyFromPassword(password.toCharArray(), salt);
                System.out.println("Creating SecretKey...");
          }else {
                System.out.println("Reusing SecretKey...");
          }

        Cipher cipher = Cipher.getInstance(ENCRYPT_ALGO);

        cipher.init(Cipher.DECRYPT_MODE, aesKeyFromPassword_S, new
GCMParameterSpec(TAG_LENGTH_BIT, iv));

        byte[] plainText = cipher.doFinal(cipherText);

        return new String(plainText, UTF_8);

    }

    // TO TEST

    public static void main(String[] args) throws Exception{

          // UAT
          String PASSWORD     =
"%+h9>GbrU~K$7WGJvD6~wF$i;Z|{mru*IxXVRyg{)l*V#Xm^2x";
          String SALT         = "sG3YrEDKPaj2nUXF";
          String IV                 = "PcGK5n4y72XL";


        String pText =
"{\"sysToken\":\"3g8MCBW*rUuTVNRTFbMRjm%3*3xYmrm\",\"action\":\"checkUserExist\",\"
email\":\"zaldyantonio@gmail.com\"}";

        EncryptorAesGcmPasswordV2 encAes = new EncryptorAesGcmPasswordV2();
        encAes.setSalt_S(SALT);
        encAes.setIv_S(IV);

encAes.setAesKeyFromPassword_S(CryptoUtils.getAESKeyFromPassword(PASSWORD.toCharArr
ay(), SALT.getBytes()));

        String encryptedTextBase64 = encAes.encrypt(pText.getBytes(UTF_8),
PASSWORD);

        // encrypt Version 2
//        for(int i=0; i < 1000; i++) {
//
   System.out.println(EncryptorAesGcmPassword.encrypt(pText.getBytes(UTF_8),
PASSWORD));
//        }

        // encrypt Version 2.1
        for(int i=0; i < 1000; i++) {
          System.out.println(encAes.encrypt(pText.getBytes(UTF_8), PASSWORD));
        }
        System.out.println("encrypt END...");
```

```
            // decrypt Version 2
//          for(int i=0; i < 1000; i++) {
//
    System.out.println(EncryptorAesGcmPassword.decrypt(encryptedTextBase64,
PASSWORD));
//          }
//          System.out.println("END...");

            // decrypt Version 2.1
            for(int i=0; i < 1000; i++) {
              System.out.println(encAes.decrypt(encryptedTextBase64, PASSWORD));
            }
            System.out.println("decrypt END...");

        }

    }
```

Sample t parameter:

JSON for checkUserExist API	Encrypted	URL Encoded
{ "sysToken":"<TO BE PROVIDED>", "action":"checkUserExist", "email":"zaldyantonio@gmail.com" }	UGNHSzVuNHk3MlhMc0czWXJFREtQYWoyblVYRiPxhTSYD+M6eaxiT2vms7SUyBbM5mZ7TqFSb7u5lzr70VRJvInSuYQdcxb/CYLCSV51AX5r4cY5Sfeqw1Q+vSpx+0bkXaS4yJquY/futtwAswDYYJB3niSnHOGASh/Jd8seWUx/NYF08N8tQn9LcXjnezoN+SDsNIQ=	UGNHSzVuNHk3MlhMc0czWXJFREtQYWoyblVYRiPxhTSYD%2BM6eaxiT2vms7SUyBbM5mZ7TqFSb7u5lzr70VRJvInSuYQdcxb%2FCYLCSV51AX5r4cY5Sfeqw1Q%2BvSpx%2B0bkXaS4yJquY%2FfuttwAswDYYJB3niSnHOGASh%2FJd8seWUx%2FNYF08N8tQn9LcXjnezoN%2BSDsNIQ%3D

* Please note that encrypted value will vary for each encryption done on the same JSON.

URL Encode encrypted value before passing as t parameter.

Sample complete api call:

https://ascott-uat.crmxs.com/?xs_app=asr_api_v2_1&t=UGNHSzVuNHk3MlhMc0czWXJFREtQYWoyblVYRiPxhTSYD%2BM6eaxiT2vms7SUyBbM5mZ7TqFSb7u5lzr70VRJvInSuYQdcxb%2FCYLCSV51AX5r4cY5Sfeqw1Q%2BvSpx%2B0bkXaS4yJquY%2FfuttwAswDYYJB3niSnHOGASh%2FJd8seWUx%2FNYF08N8tQn9LcXjnezoN%2BSDsNIQ%3D